METODOLOGÍA DE DESARROLLO CON IA

DevelopAI

Síntesis gráfica: Spec-Driven Development y orquestación multiagente

Del prompt suelto al proyecto con specs, agentes y gates de verificación.

Guías GUIA-OPENCODE.md y GUIA-HERMES.md.

Fundamentos | Arnés | Multiagente

Asignatura Programación · Tecnicatura en Desarrollo de Software

LA IDEA EN UNA FRASE

La IA ejecuta, el responsable sos vos

La potencia sin control no sirve: por eso el flujo tiene gates de aprobación

«La IA ejecuta, pero el responsable del proyecto eres tú. La habilidad que gana valor es la de fundamentos, requisitos, diseño, supervisión y evaluación; la que pierde, la sintaxis pura.»

01

Fundamentos

Arquitectura, requisitos, diseño y evaluación.

Sin esto, la IA produce ruido prolijo.

02

Agente con arnés

AGENTS.md, MEMORY.md, constitution, skills, MCPs y comandos: contexto y límites.

03

Orquestación multiagente

Coordinator, planner, implementer y reviewer reparten y controlan el trabajo.

Las tres capas son acumulativas: no se orquestan agentes sobre un proyecto sin constitution, y no se escribe una constitution sin entender el dominio.

VISIÓN GENERAL

Qué es DevelopAI: tres capas apiladas

Cada capa se apoya en la de abajo

CAPA 3 · ORQUESTACIÓN

Multiagentes

coordinator | planner | implementer | reviewer

CAPA 2 · PROCESO

Flujo SDD

spec | clarify | plan | tasks | implement | validate

CAPA 1 · CONTEXTO

Arnés del proyecto

AGENTS.md | MEMORY.md | constitution | MCPs | skills | commands

Sin la capa 1 el agente improvisa · Sin la capa 2 se escribió código que nadie pidió · Sin la capa 3 una sola persona (humana o IA) decide y nadie revisa.

CAPA 1

El arnés: 6 archivos que hacen el trabajo sucio

Se crean una vez y se mantienen en cada tarea

ESTRUCTURA FINAL DE CARPETAS

```
project/
├── AGENTS.md            instrucciones · máx 40 líneas
├── MEMORY.md            memoria entre sesiones · ~50
├── opencode.json        MCPs
├── docs/
│   └── constitution.md  6 principios innegociables
├── specs/
│   └── 001-nombre/
│       ├── spec.md      QUÉ y POR QUÉ
│       ├── plan.md      CÓMO
│       └── tasks.md     tareas verificables
├── .agents/skills/      sdd · nestjs-esm
└── .opencode/
    ├── agents/          coordinator · planner ·
    │                    implementer · reviewer
    └── commands/        /sdd-spec · /sdd-plan · …
```

AGENTS.md

Stack, comandos, convenciones, límites y verificación.

Lo primero que lee el agente.

MEMORY.md

Estado, decisiones con su porqué, aprendizajes y próximos pasos. Se actualiza por tarea.

constitution.md

6 principios innegociables. Toda spec, plan y tarea debe cumplirlos.

specs/

Una carpeta por requerimiento con spec, plan y tareas.

Es lo que se evalúa.

Regla: lo que se vuelve permanente migra de MEMORY.md a AGENTS.md; lo que es innegociable, a constitution.md.

CAPA 3

Los cuatro agentes y sus permisos

Permisos por archivo: cada agente solo puede hacer lo que su rol necesita

COORDINATOR · mode: primary

Dirige el flujo y habla con el usuario. edit ✕ shell ✕ → planner / implementer / reviewer ✓

planner

Redacta spec, plan y tareas.

Nunca escribe código.

```
edit: specs/ · docs/ · MEMORY.md
shell: ✕
```

implementer

Implementa UNA tarea.

Tests primero, después el código.

```
edit: todo (menos constitution)
shell: ✓
```

reviewer

Revisa specs y valida RF por RF.

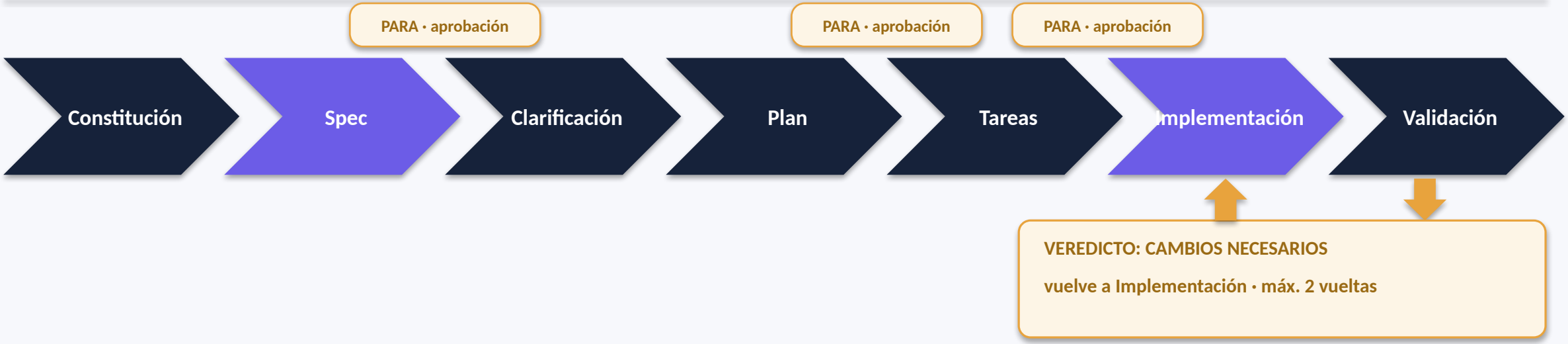
No modifica ningún archivo.

```
edit: ✕
shell: pnpm test/lint/build · git diff
```

El coordinador no escribe código y el revisor no edita archivos: así nadie valida su propio trabajo. Los subagentes NO ven la conversación: en cada llamada hay que pasarles todo el contexto.

El flujo SDD: ocho fases y gates de aprobación

Nunca se pasa de fase sin la aprobación explícita del usuario



Fase	Artefacto que queda en disco	Quién
Spec	specs/NNN/spec.md · QUÉ y POR QUÉ en EARS	planner
Clarificación	Informe QA: solo detecciones, sin soluciones	reviewer
Plan + tareas	plan.md (CÓMO) + tasks.md (RF + «Hecho cuando»)	planner
Implementación	1 commit por tarea, tests en verde, MEMORY.md	implementer
Validación	Tabla RF → test → resultado + VEREDICTO	reviewer

Cambio de requisito: /sdd-change → primero la spec, recién después el código.

FASE SPEC

La spec dice QUÉ y POR QUÉ — nunca CÓMO

Si la spec menciona librerías o archivos, está mal escrita

✔ SÍ va en la spec

- Contexto, objetivo y usuarios
- Historias de usuario (HU)
- Requisitos funcionales en EARS
- Requisitos no funcionales
- Casos límite concretos
- Fuera de alcance
- Criterios de finalización
- Dudas: [NECESITA ACLARACIÓN]

× NO va en la spec

- Framework, lenguaje o librerías
- Arquitectura y patrones a usar
- Nombres de archivos o carpetas
- Diseño de la base de datos
- Algoritmo o secuencia de pasos
- Estimación de esfuerzo
- Decisiones técnicas (van en plan.md)
- «Que sea rápido / escalable» sin número

Antes de escribir la spec, el planner hace preguntas de UNA en UNA (máximo 5). Con esas respuestas elimina la ambigüedad. Lo que no se pregunta, después se decide a dedo — y eso es exactamente lo que un reviewer detecta.

Contexto | Usuarios | HU | RF (EARS) | RNF | Casos límite | Fuera de alcance | Criterios | Dudas

REQUISITOS

EARS: requisitos que un test puede comprobar

Un requisito sin condición verificable no se puede validar

Forma	Cuándo se usa	Ejemplo
CUANDO / MIENTRAS	evento o estado continuo	CUANDO dos reservas compiten por el mismo turno, EL SISTEMA acepta UNA.
SI ... ENTONCES	condición no deseada	SI el id no existe, ENTONCES EL SISTEMA responde 404.
A MENOS QUE	excepción	A MENOS QUE el usuario sea admin, EL SISTEMA responde 403.
EL SISTEMA DEBE	requisito prefijado / no funcional	EL SISTEMA DEBE limitar limit a 100 por petición.

× «La API debe ser rápida y robusta»

No tiene condición ni resultado medible: ningún test puede decir si se cumplió. El reviewer lo devuelve como CAMBIOS NECESARIOS.

✓ «SI limit > 100, ENTONCES el sistema responde 400 con el detalle del campo»

Condición clara, resultado observable: se escribe el test antes que el código.

FASE TAREAS

Tareas pequeñas, verificables y en orden

El plan se corta en piezas que se pueden revisar en 20-30 minutos

ANATOMÍA DE UNA TAREA

```
- [ ]  T5. DTOs con validación global.        RF-4

     Hecho cuando: pnpm test:e2e pasa el test de
     datos inválidos (400) y no hay @ts-ignore.

     estimación: 25 min   ·   depende de: T4
```

Una tarea por llamada al implementer. Al terminar: un commit con mensaje `Tn: …`, la casilla `[x]` en tasks.md, MEMORY.md actualizado y —PARA—. No se empieza la siguiente hasta que el coordinador lo diga.

Orden de dependencia

T1 antes que T2 si T2 la necesita. Nada de «ya veremos».

Máximo 10 por spec

Si no entra, la spec es demasiado grande: se parte.

Tests antes que el código

El test en rojo es la prueba de que la tarea se entendió.

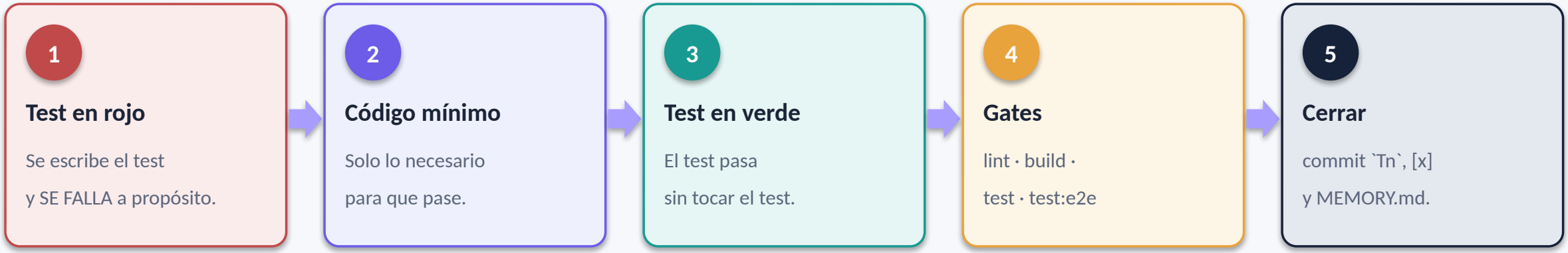
«Hecho cuando:» con comando

Si no se puede comprobar con un comando, no es verificable.

GATE

Tests como puerta: el ciclo que no se salta

Ninguna tarea se da por hecha con un gate en rojo



Si algo falla en el paso 3 o 4 → volvés al paso 1. Prohibido borrar, comentar o «arreglar» el test para que pase.

Gate	Comando	Qué detecta
Lint	pnpm lint	código muerto, tipos flojos, patrones prohibidos
Build	pnpm build	imports ESM sin .js, errores de tipos
Unit	pnpm test	la lógica de negocio y los casos límite
e2e	pnpm test:e2e	contrato real de la API: códigos de estado y payloads

FASE VALIDACIÓN

Validar es recorrer la spec RF por RF

Un RF sin test que lo cubra NO está implementado

LA SPEC

RF-1	**listado paginado**	test: e2e lista
RF-2	**404 si no existe**	test: e2e id inexistente
RF-3	**crear → 201**	test: e2e crea álbum
RF-4	**datos inválidos → 400**	test: e2e payload vacío
RF-5	**borrar → 204**	— SIN TEST

VEREDICTO: APROBADO

Todos los RF cubiertos, gates en verde, tareas con su commit y MEMORY.md al día.

VEREDICTO: CAMBIOS NECESARIOS

Se lista qué RF falla y por qué. Vuelve al implementer y después al reviewer. Máximo 2 vueltas: si sigue en rojo, se escala al usuario.

El revisor nunca edita archivos: solo ejecuta los gates y observa. Por eso su veredicto es independiente de quien planificó y de quien implementó.

COMANDOS

El flujo completo, teclado en mano

Cada comando corre con el agente que corresponde a la fase

Comando	Agente	Qué hace
/sdd-constitution	planner	Propone los 6 principios innegociables del proyecto
/sdd-spec 002-nombre idea	planner	Entrevista (máx. 5 preguntas) y genera spec.md
/sdd-clarify 002-nombre	reviewer	Revisa la spec como QA: solo detecciones
/sdd-plan 002-nombre	planner	Genera plan.md (CÓMO) — pide aprobación
/sdd-tasks 002-nombre	planner	Genera tasks.md: ≤ 10 tareas de 20-30 min
/sdd-implement 002 T3	implementer	Implementa UNA tarea, tests primero, un commit
/sdd-validate 002-nombre	reviewer	Recorre la spec RF por RF y emite VEREDICTO
/sdd-change 002 requisito	planner	Requisito nuevo: primero spec, después código
/sdd-status 002-nombre	coordinator	Fase actual, aprobaciones y siguiente paso

Atajos: TAB cambia de agente · @ referencia archivos y skills · SHIFT+TAB alterna Build/Plan · /compact libera contexto.

CAPA 1 · EXTENSIBILIDAD

MCPs y skills: contexto que se carga cuando hace falta

El contexto principal queda limpio; las instrucciones se cargan bajo demanda

MCPs ESENCIALES (opencode.json)

MCP	Función
Chrome DevTools	abre la web, lee consola, hace capturas
Context7	documentación actualizada de librerías
GitHub	repos, issues y pull requests
Supabase / Figma	bases de datos y diseños a código

QUÉ RESUELVE CADA UNO

MCP	Capacidad externa: qué el agente PUEDE hacer.
Skill	Conocimiento puntual: qué el agente SABE al hacerlo.
Comando	Proceso repetible: qué PASO hay que seguir.
Agente	Papel y permisos: QUIÉN puede hacer qué.

SKILLS (.agents/skills/.../SKILL.md)

sdd

El flujo, la plantilla de spec, las formas de EARS, el formato de tareas y los gates. Se carga siempre que se toca una spec, un plan o una tarea.

nestjs-esm

Trampas del stack: imports locales con .js, ValidationPipe global, 404 vs null, .env y ConfigModule, mock del repository en Vitest.

Instalación: crear los archivos, reiniciar OpenCode y verificar con /skills, /agents y /mcps. Si no aparecen, el agente no los va a usar — por más bien escritos que estén.

APLICACIÓN

De DevelopAI a la asignatura Programación

El Proyecto Integrador es el flujo SDD completo, con los sprints como fases

Sprint	Clase	Fase SDD	Entregable
Planning	19 · 19/10	constitution + spec	spec.md aprobada
1 · Backend	20 · 19/10	plan + tareas + T1…Tn	1 commit por tarea
2 · Auth	21 · 26/10	spec 002 → implementación	guards + 10 casos límite
3 · Testing	22 · 02/11	sdd-validate RF por RF	tabla RF → test + VEREDICTO
4 · Docker/CI	23 · 09/11	gates como workflow	lint+test bloquean el merge
5 · CD	24 · 16/11	despliegue + observabilidad	pipeline y logs
Demo	25 · 30/11	cierre y revisión	Sprint Review sobre MEMORY.md

TP-1 · De FastAPI a NestJS (Chinook)
CRUD con arquitectura por capas, spec en EARS y gates en verde.
Cronograma: Clase 3 + Clase 14.

TP-2 · Autenticación y roles (NestJS)
JWT + guards, secretos en .env y 10 casos límite con test.
Cronograma: Clase 7 + Clase 21.

CIERRE

Qué cambia en cómo se programa

GANA VALOR

- Fundamentos y arquitectura
- Requisitos, specs y casos límite
- Diseño y separación de capas
- Supervisión: leer y cuestionar
- Evaluación: qué es correcto

PIERDE VALOR

- Sintaxis de memoria
- Copiar–pegar sin entender
- Escribir código sin requisitos
- «Funciona en mi máquina»
- Validar a ojo, sin tests

> Regla de oro: la IA ejecuta, pero el responsable del proyecto sos vos. Si no podés explicar por qué el código hace lo que hace — contra la spec y contra la constitution — todavía no está terminado.